

Inhaltsverzeichnis

AB1-3 LERNZIELE	2
AB5 GIT	3
<i>Verwendung eines Repo im Team und für Lehrperson verfügbar</i>	<i>3</i>
<i>Arbeitsschritte als Commit sichtbar (in Dokumentation mit 2-3 Beispielen belegt).....</i>	<i>3</i>
<i>Sinnvolle Nutzung von Branches mit branch und merge (in Dokumentation mit 2-3 Beispielen belegt)</i>	<i>3</i>
<i>Sinnvolles readme.md für das Projekt erstellt</i>	<i>4</i>
<i>Exp: Nutzung von Merge Requests</i>	<i>4</i>
<i>Exp: Anwendung von GitFlow / Lean</i>	<i>4</i>
<i>Exp: Dokumentation eines Spezialfalles (Revert, Konflikt, Rebase usw.)</i>	<i>4</i>
AB6 PLANUNG	5
BLACKBOX	5
<i>Beschreibung der Vorbedingungen (Notwendige Eingaben, Optionen, Benutzerinterface) und</i>	
<i>Nachbedingungen (Erwartetes Resultat nach der Ausführung des Programms)</i>	<i>5</i>
WHITEBOX.....	7
<i>Ablaufplanung alles vorhanden und verständlich dargestellt</i>	<i>7</i>
<i>Programmtechnische Teilprobleme aufgezeigt und beschrieben</i>	<i>11</i>
<i>Teilprobleme Schätzungen für Komplexität und Aufwand vorhanden.....</i>	<i>11</i>
<i>Reihenfolge (Priorität) der Umsetzung der Teilprobleme mit Begründung vorhanden</i>	<i>14</i>
<i>Exp: Architektur als Klassendiagramm vorhanden</i>	<i>16</i>
<i>Exp: Realistisches Mockup erstellen.....</i>	<i>21</i>
<i>Exp: Zustände der Masken geplant.....</i>	<i>21</i>
ÜBERGEORDNETE THEMEN.....	26
<i>Einsatz von Komponenten von Drittherstellern wird dokumentiert (AB1)</i>	<i>26</i>
<i>Dateien lesen und schreiben (txt, csv, json) (AB3).....</i>	<i>26</i>
<i>Testing mittels UnitTest (AB4).....</i>	<i>28</i>
<i>Clean Code Regeln werden eingehalten und anhand eines Beispiels dokumentiert (AB4)</i>	<i>28</i>
<i>Fehlerhandling (try catch) sinnvoll eingesetzt.....</i>	<i>28</i>
<i>API Dokumentation (DoxFX, Sandcastle, DoxyGen)</i>	<i>32</i>

AB1-3 Lernziele

Selbstdeklaration erreichter Lernziele «Basic»:

AB	Thema	Vorhanden
AB1	Maske mit statischem Layout und Steuerelementen erstellen	
AB1	GUI Events handeln und Daten in GUI zurückschreiben	
AB1	Feedback ergonomisch gestalten	
AB1	Komplette Maske mit sauberen Zuständen und Validierung der Eingaben erstellen und testen	
AB1	Mockup einer Anwendung erstellt mit min. 3 Masken und Navigation	
AB2	Eine Klasse als Tabelle erstellen	
AB2	Fremdschlüsselbeziehungen als Tabellen erstellen	
AB2	Abfragen erstellen	
AB2	Datenmanipulationen durchführen	

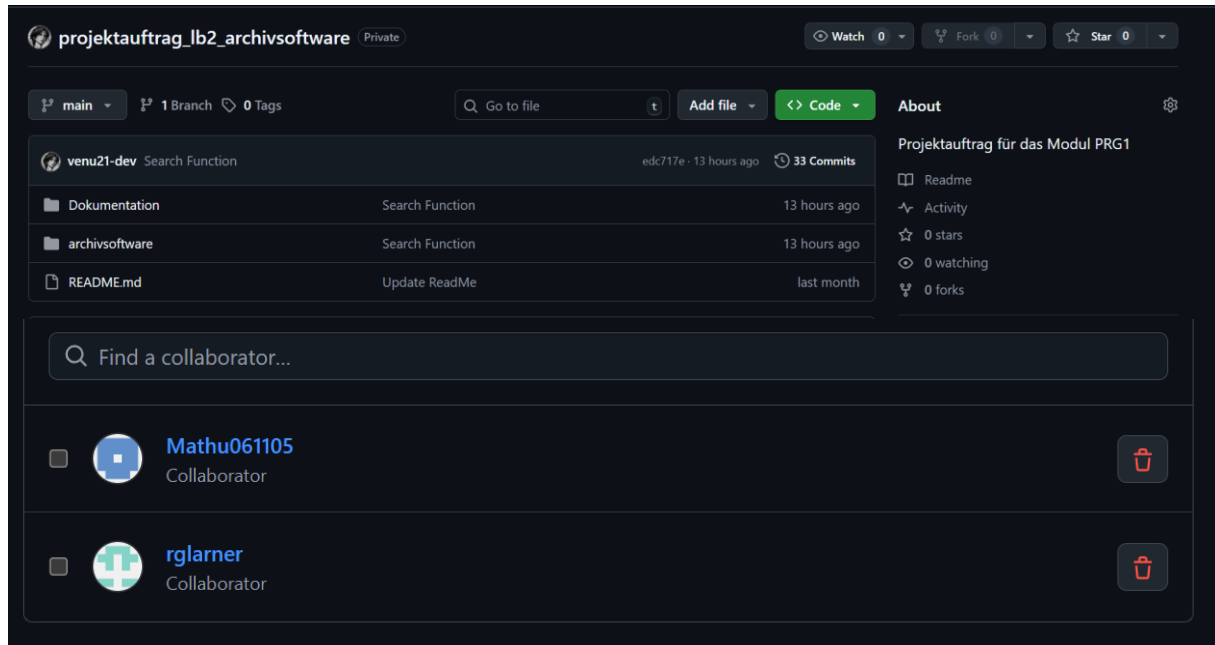
Selbstdeklaration erreichter Lernziele «Expert»:

AB	Thema	Vorhanden
AB1	Non-blocking Calls ausführen im Hintergrund	
AB1	Einen Timer für Polling verwenden	
AB1	Benutzersteuerelemente sinnvoll einsetzen	
AB1	Datenbindung z.B. DataGrid zur Anzeige	
AB2	Datenzugriff mit Pattern (Repository oder Serviceklasse)	
AB2	saubere Architektur (eigenes Projekt, Onion, DI, ..)	
AB2	verschlüsselte Kommunikation mit der DB	
AB2	Benutzerfreundliche mandantfähige Lösung (DB-Switch)	

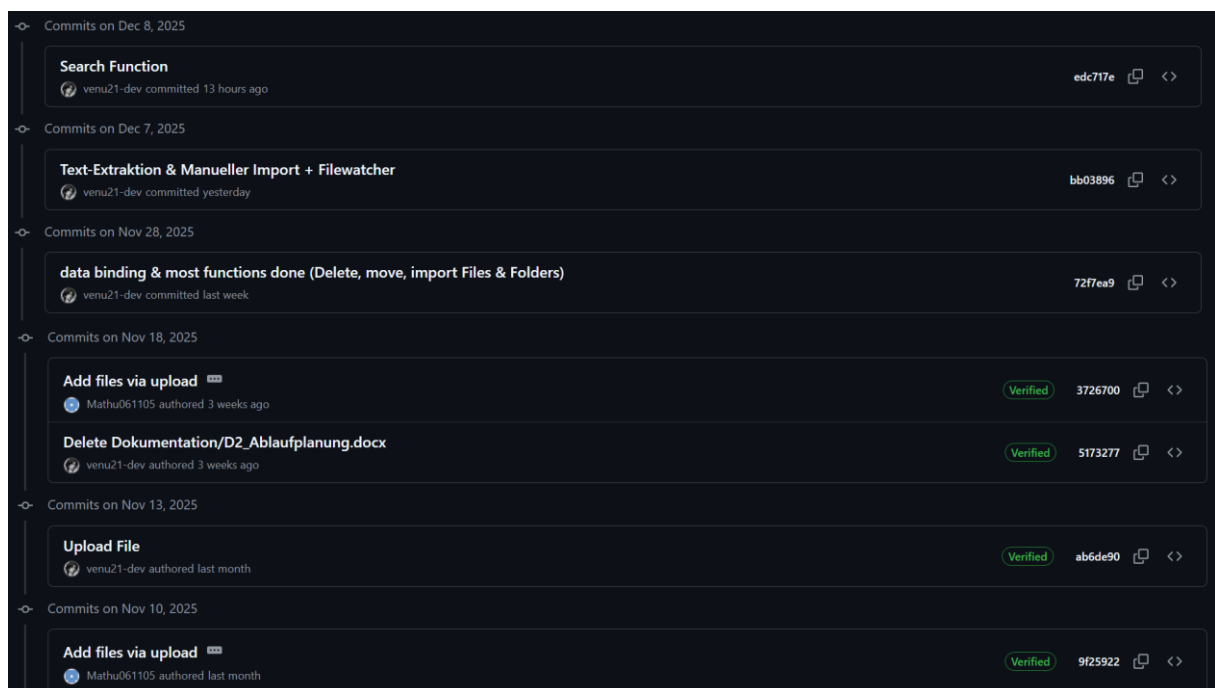
AB5 Git

Verwendung eines Repo im Team und für Lehrperson verfügbar

- https://github.com/venu21-dev/projektauftrag_lb2_archivsoftware



Arbeitsschritte als Commit sichtbar (in Dokumentation mit 2-3 Beispielen belegt)



Sinnvolle Nutzung von Branches mit branch und merge (in Dokumentation mit 2-3 Beispielen belegt)

Sinnvolles readme.md für das Projekt erstellt

	README
	Archivsoftware - Dokumentenverwaltung
Desktop-Anwendung zur automatischen Ablage, Verwaltung und Suche von Dokumenten in einer SQL-Server-Datenbank	
	Projektübersicht
Projektname: Archivsoftware Team: Jegatheeswaran Mathumithan & Manivannan Venurshan Kurs: PROG1 - LB2 Technologien: C# WPF, Entity Framework Core, SQL Server	
Zielsetzung	
Entwicklung einer Desktop-Anwendung, die eine automatische Ablage, Verwaltung und Volltextsuche von PDF- und DOCX-Dokumenten in einer SQL-Server-Datenbank ermöglicht. Der Fokus liegt auf GUI-Usability, Datenbankankbindung via Entity Framework Core, einer sauberen Schichtenarchitektur und hoher Codequalität.	
Hauptfunktionen	
Ordnerverwaltung	
- Hierarchische Ordnerstruktur (anlegen, umbenennen, verschieben, löschen) - Validierung: Eindeutige Namen pro Ebene - Intuitive Baumansicht mit auf-/zuklappbaren Unterordnern	
Dateiimport	
- Automatisch: Überwachter Ordner (FileSystemWatcher) - Manuell: Import per GUI (FileDialog) - Unterstützte Formate: PDF, DOCX - Speicherung als BLOB + extrahierter PlainText in Datenbank	
Volltextsuche	
- Suche über extrahierten PlainText (SQL LIKE) - Anzeige der Treffer mit: - Dokumenttitel - Ordnerpfad - Text-Snippet mit Highlight des Suchbegriffs - Echtzeit-Filterung der Suchergebnisse	
Benutzeroberfläche	
- Links: Ordnerbaum (TreeView) zur Navigation - Mitte: Suchergebnisse / Dokumentenliste (ListView) - Rechts: Detailsansicht des ausgewählten Dokuments - Dunkles, modernes Design für angenehmes Arbeiten	
Architektur	
Das Projekt folgt einer 3-Schichten-Architektur:	
<pre>ArchivSoftwareApp/ ├── Presentation/ │ ├── MainWindow.xaml │ ├── ViewModels/ │ └── Controls/ │ └── Custom Controls │ ├── Business/ │ ├── Services/ │ │ ├── DocumentService.cs │ │ ├── FolderService.cs │ │ └── SearchService.cs │ └── Models/ │ └── Business Models │ ├── DataAccess/ │ ├── Entities/ │ │ ├── Folder.cs │ │ └── Document.cs │ ├── Repositories/ │ │ ├── FolderRepository.cs │ │ ├── FolderRepository.cs │ │ ├── DocumentRepository.cs │ │ └── DocumentRepository.cs │ └── DbContext.cs │ └── GUI Layer (WPF) ├── MainFunction ├── MVVM ViewModels └── Custom Controls Business Logic Layer Business Services Database Entities Repository Pattern DbContext Core Context</pre>	

Technische Stack

Frameworks & Libraries

- .NET 8 - Application Framework
- WPF - Windows Presentation Foundation für GUI
- Entity Framework Core 7.0+ - ORM für Datenbankszugriff
- SQL Server Express LocalDB - Datenbank

NuGet-Pakete

```
<packageReference Include="Microsoft.EntityFrameworkCore" Version="7.0.*" />  
<packageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="7.0.*" />  
<packageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="7.0.*" />  
<packageReference Include="ITextSharp" Version="5.5.*" />  
<packageReference Include="DocumentFormat.OpenXml" Version="2.20.*" />
```

Entwicklungstools

- Visual Studio 2022 - IDE
- Git / GitHub - Versionsverwaltung
- SQL Server Management Studio (SSMS) - Datenbankverwaltung

Features

Ordner erstellen

- Klicke auf "+ Neuer Ordner"
- Gib einen eindeutigen Namen ein
- Ordner erscheint in der Ordnerstruktur links

Dokument importieren

Manuell:

- Klicke auf "Importieren"
- Wähle eine PDF- oder DOCX-Datei
- Dokument wird in den aktuell ausgewählten Ordner importiert

Automatisch:

- Konfiguriere den Überwachungsordner (In Settings)
- Kopiere PDF/DOCX in den Überwachungsordner
- Dokument wird automatisch importiert

Dokument suchen

- Gib ein Stichwort in das Suchfeld oben ein
- Drücke Enter oder klicke auf Suchen
- Ergebnisse werden in der Mitte angezeigt mit Snippet
- Klicke auf ein Ergebnis → Dokument öffnet sich rechts

Dokument anzeigen

- Klicke auf ein Dokument in der Liste oder im Baum
- Dokumenteninhalt wird rechts angezeigt
- Suchbegriffe werden gelb markiert (bei aktiver Suche)

Lernziele (LB2)

Dieses Projekt erfüllt folgende Lernziele:

Basic (14 Lernziele)

- A4: Komplette Maske mit Validierung und Fehlerbehandlung
- B1: Klassen als Tabellen erstellen
- B2: Fremdschlüssel-Beziehungen
- B3: SQL Abfragen erstellen
- B4: Datenmanipulationen durchführen
- C1: Git Repository im Team
- C2: Ausgewählte Commits
- C4: Sinnvolles README.md
- D1: Vor-/Nachbedingungen (Testing)
- D2: Ablaufplanung
- D3: Teilprobleme beschrieben
- D4: Schätzungen vorhanden
- E1: NuGet-Pakete von Drittherstellern
- E4: Clean Code Regeln einhalten

Exp: Nutzung von Merge Requests

Exp: Anwendung von GitFlow / Lean

Exp: Dokumentation eines Spezialfalles (Revert, Konflikt, Rebase usw.)

AB6 Planung

Blackbox

Beschreibung der Vorbedingungen (Notwendige Eingaben, Optionen, Benutzerinterface) und Nachbedingungen (Erwartetes Resultat nach der Ausführung des Programms)

VORBEDINGUNGEN

Was muss gegeben sein, damit das Programm erfolgreich eingesetzt werden kann?

Systemvoraussetzungen:

- Windows 10/11 (64-bit)
- .NET 8.0 Runtime
- SQL Server Express LocalDB
- 4 GB RAM
- 500 MB Festplattenspeicher

Umgebung / Infrastruktur:

- Lokale Datenbank (SQL Server LocalDB)
- Dateisystem-Zugriff für Watcher-Ordner
- Lese-/Schreibrechte für Dateien

Notwendige Eingaben:

- Pflicht: Keine - Programm startet ohne Parameter
- Optional: Watcher-Ordner-Pfad (Standard: C:\ArchiveWatch\)

Benutzerinterface:

- GUI-basiert (WPF Desktop-Anwendung)
- 3-Spalten-Layout: Ordnerstruktur, Suchergebnisse, Dokumentenvorschau
- Suchfeld + Buttons ("Neuer Ordner", "Importieren")

Systemgrenze:

Gehört zum Projekt:

- Desktop-Anwendung (WPF)
- Datenbank (SQL Server LocalDB)
- Text-Extraktion (PDF/DOCX)
- FileSystemWatcher

Wird extern geliefert:

- PDF/DOCX Dateien (vom Benutzer)
- .NET Runtime, SQL Server LocalDB
- NuGet-Bibliotheken (iTextSharp, OpenXml)

NACHBEDINGUNGEN

Welche Veränderungen werden durch die Ausführung des Programms hervorgerufen? Das Programm ist eine Desktop-Anwendung zur Dokumentenverwaltung. Es verwaltet PDF/DOCX-Dateien in einer Datenbank mit Volltextsuche. Die Veränderungen betreffen die Datenbank, das Dateisystem bleibt unverändert.

Nach Programmstart:

Desktop-Anwendung öffnet sich, lädt Ordnerstruktur aus Datenbank und startet FileSystemWatcher.

Nach Dokument-Import:

Datei wird als BLOB in Datenbank gespeichert, Text extrahiert und Dokument erscheint in Ordnerstruktur.

Beispiel:

- "Rechnung_2024_Q3.pdf" wird importiert
- In DB als BLOB gespeichert
- Text extrahiert: "Rechnung Nr. 2024... CHF 15'450.00..."
- Sichtbar in Ordner "Rechnungen"

Nach Ordner-Verwaltung:

Ordner erstellen: Ordner erscheint in TreeView, Eintrag in Datenbank.

Ordner umbenennen: Name in Datenbank aktualisiert, TreeView aktualisiert sich.

Ordner löschen: Leerer Ordner wird gelöscht.

Bei Ordner mit Dokumenten: Warnung anzeigen, bei OK wird Ordner samt Dokumenten gelöscht.

Nach Suche:

Volltextsuche liefert Ergebnisse mit Dateiname, Ordner-Pfad und Text-Snipppet.

Beispiel:

Benutzer sucht "15450"

3 Treffer gefunden

Liste zeigt:

- Rechnung_2024_Q3.pdf (Archiv / 2024)

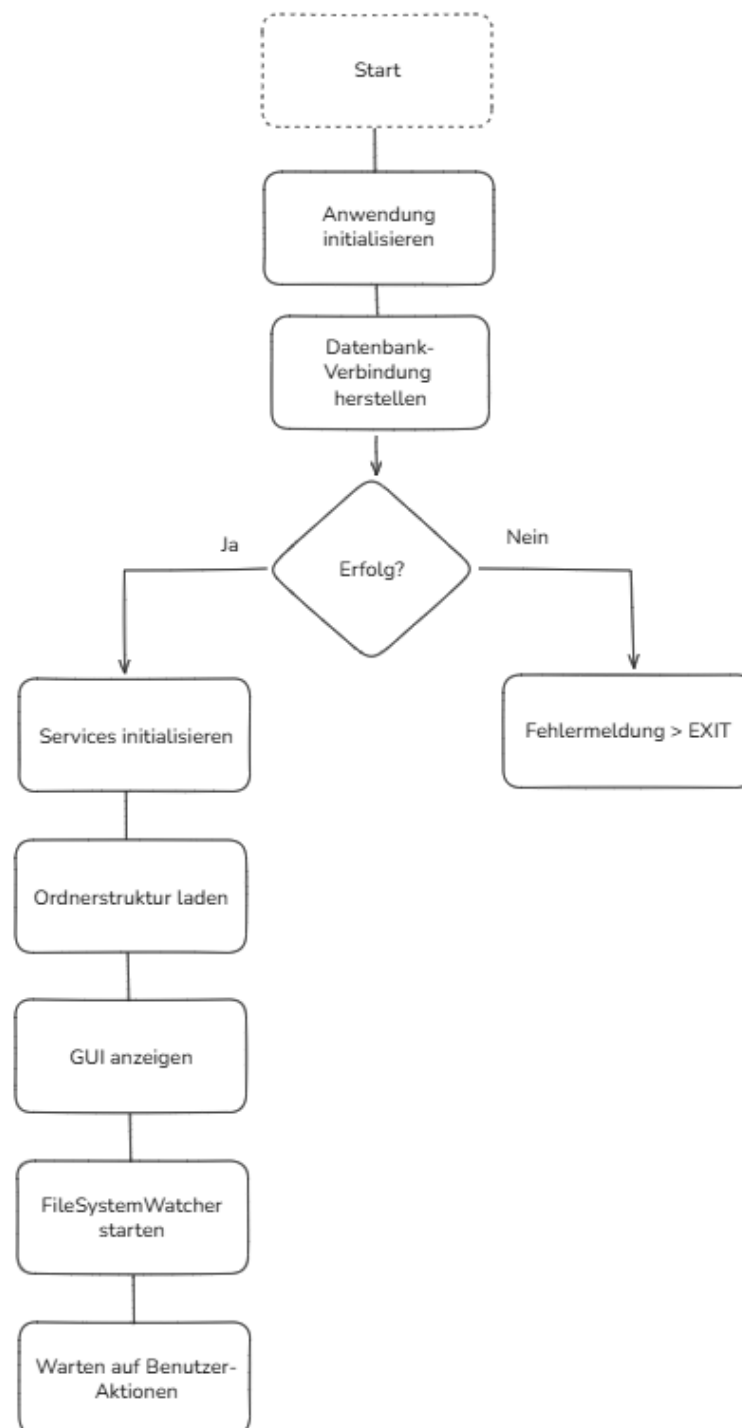
Whitebox

Ablaufplanung alles vorhanden und verständlich dargestellt

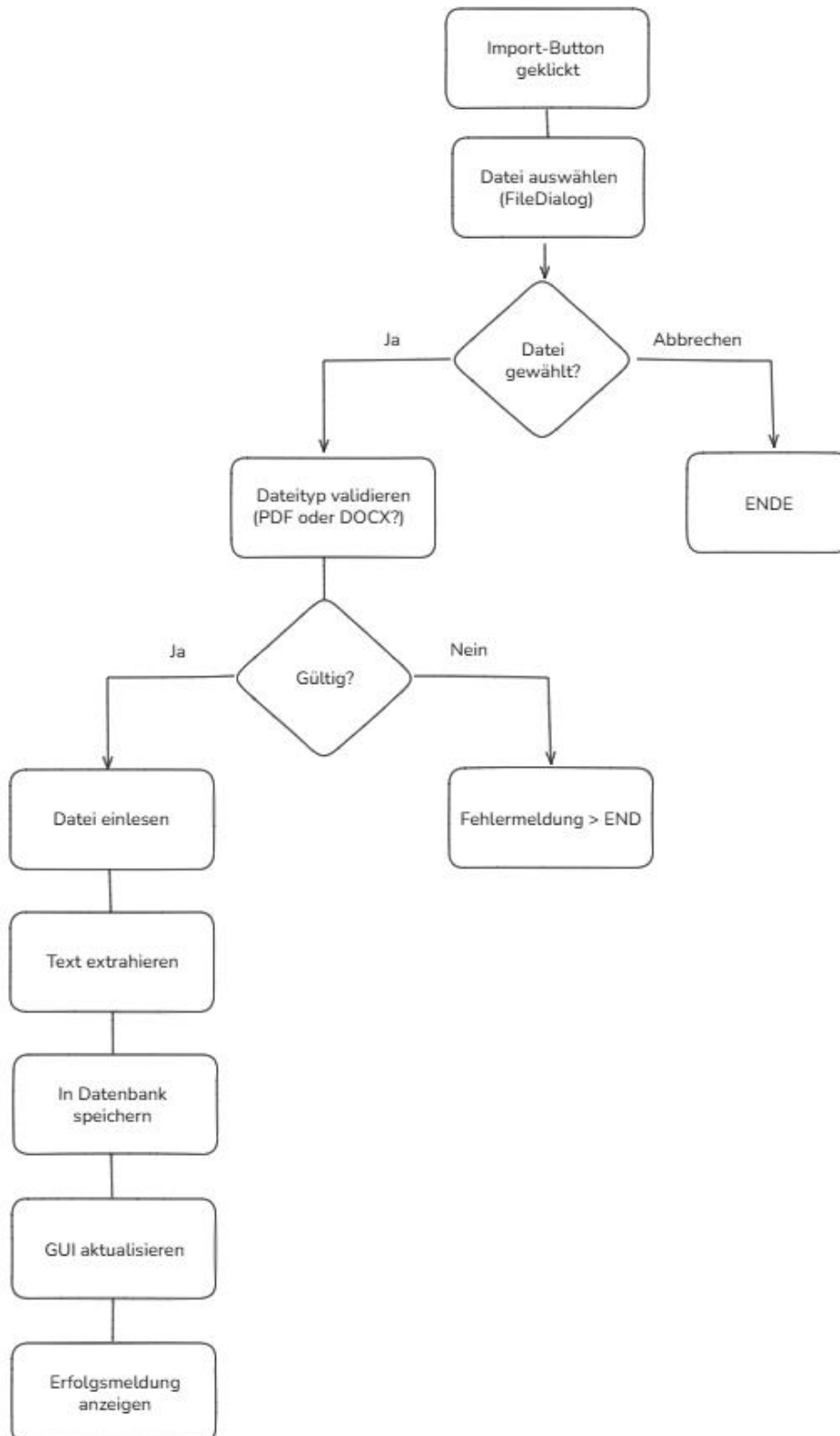
PROGRAMM ABLAUF PLAN

Die Ablaufplanung zeigt die Hauptschritte des Programms als Flussdiagramme. Die Darstellung fokussiert sich auf die wichtigsten Funktionen der Anwendung.

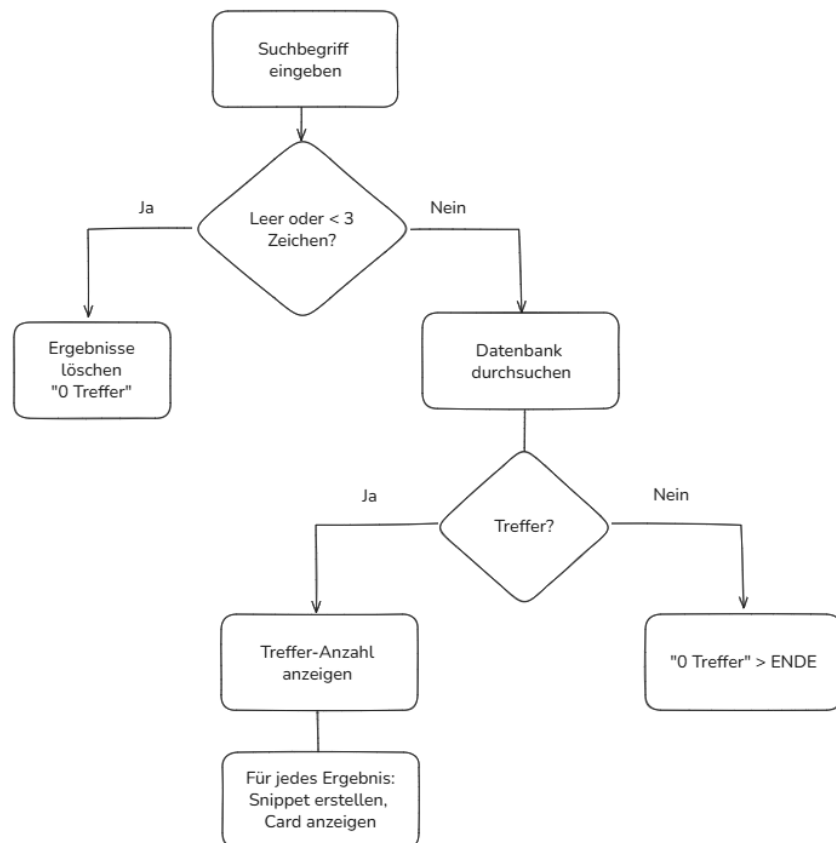
PROGRAMMSTART & INITIALISIERUNG



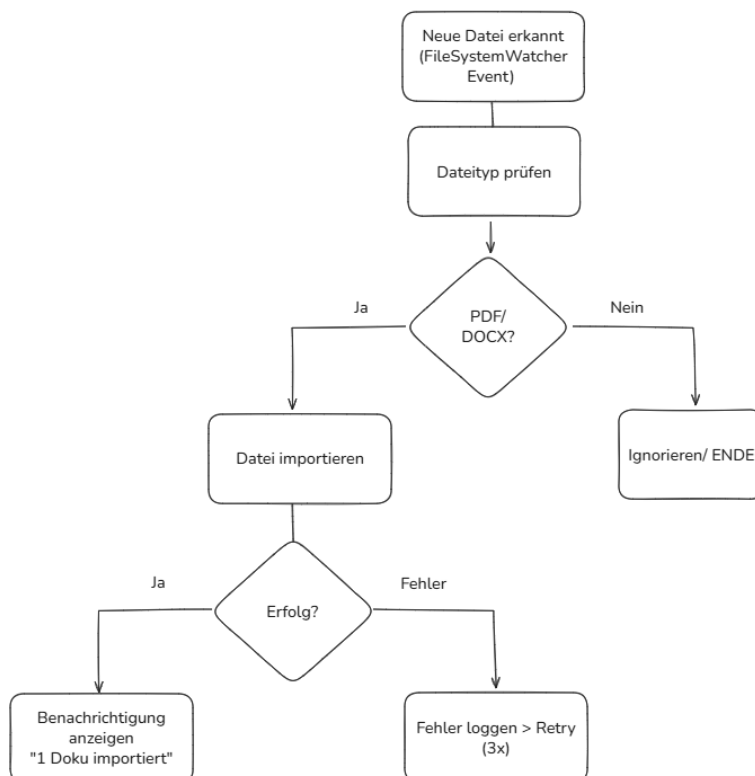
DOKUMENT IMPORTIEREN



SUCHE DURCHFÜHREN

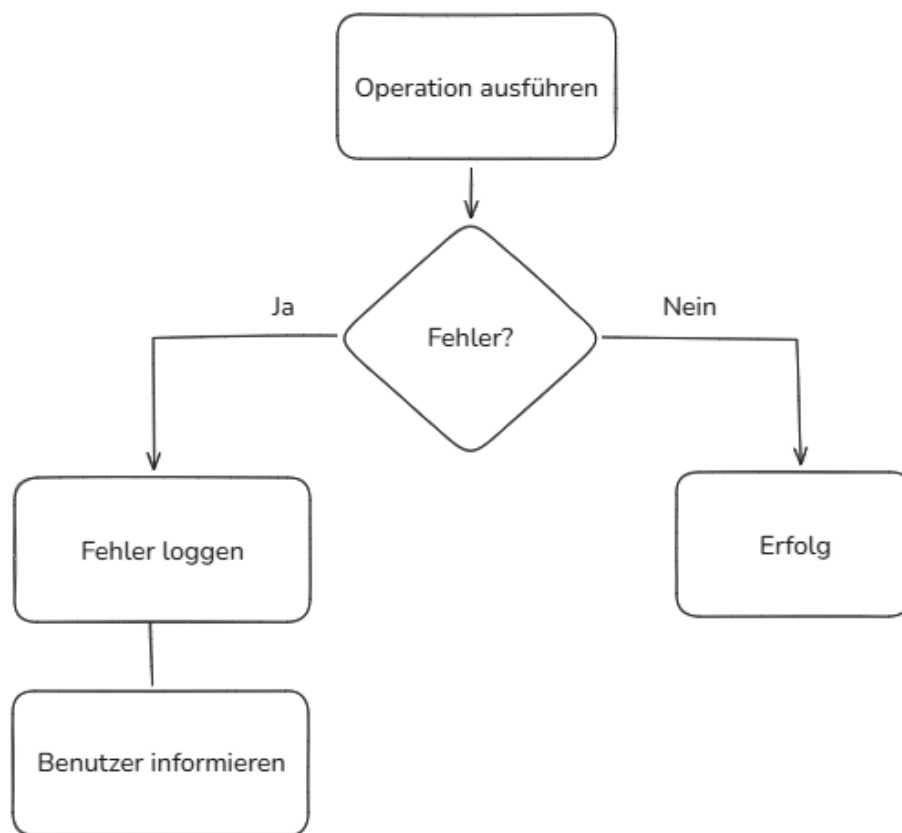


AUTOMATISCHER IMPORT (FileSystemWatcher)



FEHLERBEHANDLUNG

Alle Operationen verwenden Try-Catch-Blöcke:



Programmtechnische Teilprobleme aufgezeigt und beschrieben & Teilprobleme Schätzungen für Komplexität und Aufwand vorhanden

Anzahl identifizierte Teilprobleme: 8

Bewertungs-Skala:

- **Komplexität:** 1 sehr leicht bis 5 sehr schwer
- **Aufwand:** Geschätzte Stunden

TEILPROBLEM 1: Hierarchische Ordnerstruktur

Ordner können Unterordner enthalten > Eine Folder-Tabelle muss auf sich selbst referenzieren

Warum das ein Problem ist:

- Self-Referencing in Entity Framework ist komplex
- ParentFolderId als Foreign Key zu eigener Tabelle
- Navigation Properties (SubFolders, ParentFolder) müssen richtig konfiguriert werden

Geschätzter Aufwand: 3 Stunden

Komplexität: 4/5 (Schwer)

Begründung Komplexität & Aufwand:

Haben Self-Referencing noch nie gemacht. Tutorials zeigen oft andere Ansätze. 1h für Recherche, 1h für Implementation, 1h für Testing und Debugging der Beziehungen.

TEILPROBLEM 2: PDF Text-Extraktion mit alter Library-Version

Text aus PDF extrahieren mit iTextSharp 5.5.13.3

Warum das ein Problem ist:

- Neuere Version (iText7) ist kostenpflichtig
- API ist komplett anders zwischen den Versionen
- Namespace: iTextSharp.text.pdf (alt) vs. iText.Kernel.Pdf (neu)
- Encoding-Probleme möglich (UTF-8 vs. Windows-1252)

Geschätzter Aufwand: 4 Stunden

Komplexität: 5/5 (Sehr schwer)

Begründung Komplexität & Aufwand:

Kritisches Kern-Feature. Alte Library-Version mit wenig Dokumentation. 2h Recherche, 1h Implementation, 1h Testing mit verschiedenen PDF-Dateien.

TEILPROBLEM 3: WPF TreeView mit DataBinding

Hierarchische Ordnerstruktur im TreeView anzeigen wie Windows Explorer

Warum das ein Problem ist:

- WPF ist relativ neu für uns
- HierarchicalDataTemplate ist komplexes Konzept
- DataBinding zwischen ViewModel und View
- Auf-/Zuklappen Logik

Geschätzter Aufwand: 4 Stunden

Komplexität: 4/5 (Schwer)

Begründung Komplexität & Aufwand:

TreeView ist eines der komplexesten WPF Controls. Viele Online-Beispiele sind für MVVM, wir machen aber Code-Behind. DataBinding-Syntax ist neu. 1h WPF TreeView Grundlagen lernen, 2h XAML Code schreiben und debuggen, 1h DataBinding zum Laufen bringen.

TEILPROBLEM 4: FileSystemWatcher für automatischen Import

Ordner überwachen und neue PDF/DOCX automatisch importieren

Warum das ein Problem ist:

- Filter auf *.pdf und *.docx gleichzeitig
- Exception Handling bei Import-Fehlern

Geschätzter Aufwand: 3 Stunden

Komplexität: 5/5 (Sehr schwer)

Begründung Komplexität & Aufwand:

FileSystemWatcher ist bekannt für Bugs. 1h Grundlagen lernen, 1h Implementation, 1h Testing mit echten Dateien

TEILPROBLEM 5: DOCX Text-Extraktion mit OpenXml

Text aus Word-Dokumenten (.docx) extrahieren

Warum das ein Problem ist:

- OpenXml SDK ist riesige Library mit hunderten Klassen
- Namespace: DocumentFormat.OpenXml.Packaging komplex

Geschätzter Aufwand: 3 Stunden

Komplexität: 4/5 (Schwer)

Begründung Komplexität & Aufwand:

OpenXml Dokumentation ist komplex. Word-Dateien können sehr unterschiedlich strukturiert sein. Nicht so schwer wie PDF. 1h Recherche (OpenXml Tutorial), 1h Implementation, 1h Testing.

TEILPROBLEM 6: Validierung eindeutige Ordnernamen pro Ebene

"Rechnungen" darf nicht 2x auf gleicher Ebene existieren, aber in verschiedenen Parent-Ordern schon

Warum das ein Problem ist:

- Muss VOR dem Speichern geprüft werden
- UI muss Fehler schön anzeigen

Geschätzter Aufwand: 2 Stunden

Komplexität: 3/5 (Mittel)

Begründung Komplexität & Aufwand:

LINQ kennen wir teilweise. Logik ist klar, aber Umsetzung braucht Nachdenken. 0.5h Validierungs-Logik schreiben, 0.5h in Service integrieren, 1h Testing

TEILPROBLEM 7: Storage Performance bei grossen Dateien

PDF/DOCX als byte in SQL Server speichern - Performance bei grossen Dateien?

Warum das ein Problem ist:

- Grosse Dateien (>10MB) können Datenbank verlangsamen
- Laden von vielen Dokumenten könnte langsam sein
- Alternative: Dateien im Filesystem, nur Pfad in DB?

Geschätzter Aufwand: 2 Stunden

Komplexität: 3/5 (Mittel)

Begründung Komplexität & Aufwand:

Performance-Testing ist einfach. Falls zu langsam, gibt's klare Alternativen. 1h Implementation, 0.5h Performance testen (10-20 Dokumente), 0.5h Optimierung falls nötig.

TEILPROBLEM 8: Suchbegriff in Vorschau gelb markieren

In der Dokument-Vorschau (rechts) soll der Suchbegriff gelb highlighted sein

Warum das ein Problem ist:

- TextBox kann Text nicht farbig markieren
- String-Manipulation: Suchbegriff finden, Text splitten, mit <Run Background="Yellow"> neu zusammenbauen
- Bei mehreren Vorkommen: Alle markieren oder nur erste?

Geschätzter Aufwand: 2 Stunden

Komplexität: 3/5 (Mittel)

Begründung Komplexität:

RichTextBox ist komplexer als TextBox. FlowDocument Syntax ist neu.

1h RichTextBox/FlowDocument lernen, 0.5h Highlight-Logik schreiben, 0.5h Testing.

Reihenfolge (Priorität) der Umsetzung der Teilprobleme mit Begründung vorhanden

ÜBERSICHT TEILPROBLEME

Nr	Teilproblem	Komplexität	Aufwand
1	Hierarchische Ordnerstruktur	4/5	3h
2	PDF Text-Extraktion	5/5	4h
3	WPF TreeView	4/5	4h
4	FileSystemWatcher	5/5	3h
5	DOCX Text-Extraktion	4/5	3h
6	Validierung Ordernamen	3/5	2h
7	BLOB Storage Performance	3/5	2h
8	Suchbegriff Highlight	3/5	2h

REIHENFOLGE DER UMSETZUNG

1. Hierarchische Ordnerstruktur

Basis für alles. Ohne Ordner-System können keine Dokumente zugeordnet werden. Entity Framework Self-Referencing muss funktionieren bevor wir weitermachen.

2. Validierung Ordernamen

Warum jetzt? Direkt nach Ordner-Struktur, damit von Anfang an saubere Daten vorhanden sind. LINQ-Query ist einfacher wenn Grundstruktur steht.

3. WPF TreeView

Warum jetzt? GUI brauchen wir zum Testen. Wir können nur prüfen, ob Ordner-Struktur funktioniert wenn wir sie sehen können.

4. PDF Text-Extraktion

Schwerstes Problem früh angehen. iTextSharp 5.5.13.3 ist kritisch - wenn das nicht funktioniert, müssen wir Alternative suchen. Braucht funktionierende Ordner-Struktur zum Testen.

5. DOCX Text-Extraktion

Warum jetzt? Direkt nach PDF, weil beide ähnlich sind. OpenXml ist moderner als iTextSharp - deshalb zweite Priorität.

6. BLOB Storage Performance

Warum jetzt? Müssen prüfen ob grosse Dateien (>10MB) funktionieren so lange noch Zeit ist - nicht erst am Schluss merken dass grosse PDFs crashen.

7. Suchbegriff Highlight

Warum jetzt? Text-Extraktion funktioniert, jetzt Suche implementieren. Benötigt PlainText aus Datenbank.

8. FileSystemWatcher

Warum zuletzt? Braucht alles andere! Watcher muss Import auslösen, Import braucht Text-Extraktion, Text-Extraktion braucht Ordner-Struktur. FileSystemWatcher ist bekannt für Bugs - deshalb viel Zeit für Testing am Ende.

BEGRÜNDUNG

Bottom-Up Ansatz:

Erst Datenbank-Fundament (Ordner-Struktur), dann GUI zum Testen (TreeView), dann Kern-Features (Text-Extraktion), zuletzt Automatisierung (FileSystemWatcher).

Risikomanagement:

Schwierige Probleme früh angehen (PDF mit alter Library, Self-Referencing in EF Core). Falls unlösbar bleibt noch Zeit für Alternativen. Einfache Features am Ende (Highlight, Validierung).

Abhängigkeiten:

Ordner-Struktur > TreeView (zum Anzeigen) > Text-Extraktion (braucht Ordner-Zuordnung) > Suche (braucht extrahierten Text) > FileSystemWatcher (braucht alles).

Testing-Möglichkeit:

Nach jedem Schritt können wir testen, ob es funktioniert. Nach TP3 sehen wir Ordner, nach TP5 können wir Dokumente importieren, nach TP7 können wir suchen, nach TP8 funktioniert alles automatisch.

Exp: Architektur als Klassendiagramm vorhanden

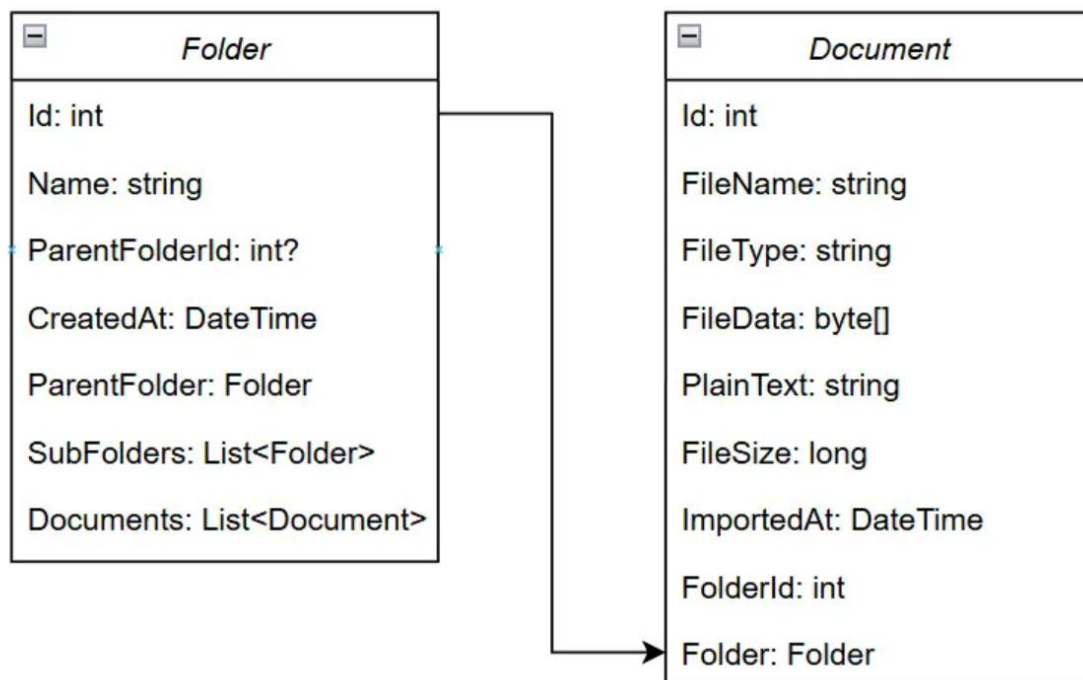
Architektur: 3-Schichten-Modell

Anzahl Klassen: 11

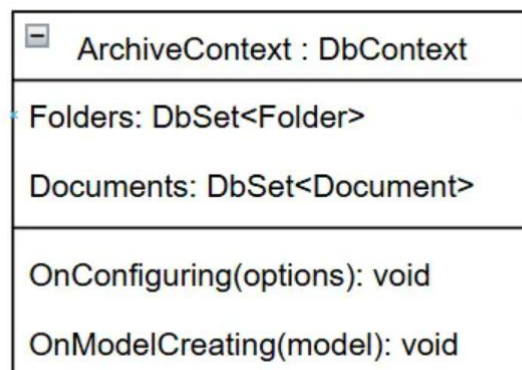
Schichten:

1. Entities / Models (2 Klassen)
2. Data Access Layer (1 Klasse)
3. Business Logic Layer (4 Klassen)
4. Utilities (2 Klassen)
5. Presentation Layer (2 Klassen)

1. ENTITIES / MODELS



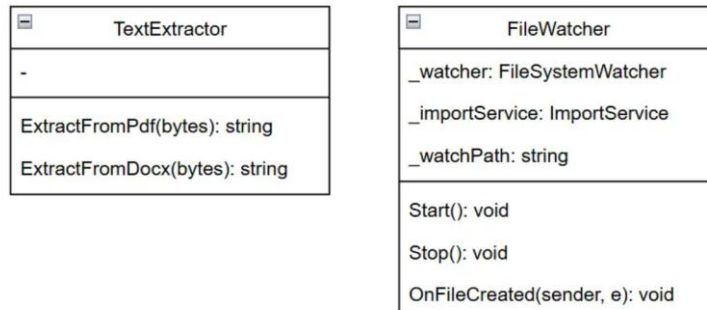
2. DATA ACCESS LAYER



3. BUSINESS LOGIC LAYER – SERVICES



4. UTILITIES



5. PRESENTATION LAYER – UI



KLASSENBESCHREIBUNGEN

Folder

Kerndaten: Ordner-Informationen (Name, Hierarchie, Erstellungsdatum)

Kernfunktionalität: Speichert hierarchische Ordnerstruktur mit Self-Referencing

Document

Kerndaten: Dokument-Metadaten (Name, Typ, Grösse, Datum) + BLOB (Datei-Inhalt) + PlainText (extrahierter Text)

Kernfunktionalität: Speichert Dokumente als BLOB mit Volltextsuche-Vorbereitung

ArchiveContext

Kerndaten: DbSet für Folders und Documents

Kernfunktionalität: Entity Framework DbContext - Verbindung zur SQL Server Datenbank, konfiguriert Beziehungen und Constraints

FolderService

Kerndaten: Referenz zu ArchiveContext

Kernfunktionalität: CRUD-Operationen für Ordner, lädt Root-Ordner und Unterordner

DocumentService

Kerndaten: Referenz zu ArchiveContext

Kernfunktionalität: CRUD-Operationen für Dokumente, lädt Dokumente nach Ordner

SearchService

Kerndaten: Referenz zu ArchiveContext

Kernfunktionalität: Volltextsuche über PlainText-Spalte, sucht in allen Dokumenten oder nur in einem Ordner

ImportService

Kerndaten: Referenz zu ArchiveContext und TextExtractor

Kernfunktionalität: Importiert Dokumente (PDF/DOCX), extrahiert Text, speichert in Datenbank, validiert Dateitypen

TextExtractor

Kerndaten: Keine

Kernfunktionalität: Extrahiert Text aus PDF (iText7) und DOCX (OpenXML) Dateien

FileWatcher

Kerndaten: FileSystemWatcher + Pfad + ImportService

Kernfunktionalität: Überwacht Ordner, importiert neue PDF/DOCX automatisch

MainWindow

Kerndaten: Referenzen zu allen Services

Kernfunktionalität: GUI - zeigt Ordner-Tree, Dokument-Liste, Suche, Import-Button, Preview

TESTFÄLLE

Folder

Testfall 1: Root-Ordner erstellen

Testdaten: Name = "Rechnungen", ParentFolderId = NULL

Erwartung: Ordner wird in DB gespeichert, Id wird generiert

Testfall 2: Unterordner erstellen

Testdaten: Name = "2024", ParentFolderId = 1

Erwartung: Ordner wird mit Referenz zu Parent gespeichert

Document

Testfall 1: PDF importieren

Testdaten: FileName = "Rechnung.pdf", FileType = "PDF", FileData = [bytes], FolderId = 1

Erwartung: Dokument wird mit BLOB gespeichert, PlainText extrahiert

Testfall 2: DOCX importieren

Testdaten: FileName = "Brief.docx", FileType = "DOCX", FileData = [bytes], FolderId = 1

Erwartung: Dokument wird mit BLOB gespeichert, PlainText extrahiert

FolderService

Testfall 1: GetRootFolders()

Testdaten: Mehrere Ordner in DB (einige mit ParentFolderId = NULL)

Erwartung: Nur Root-Ordner werden zurückgegeben

Testfall 2: CreateFolder()

Testdaten: Folder mit Name = "Test"

Erwartung: Ordner wird in DB gespeichert, SaveChanges() erfolgreich

DocumentService

Testfall 1: GetDocumentsByFolder(id)

Testdaten: FolderId = 5

Erwartung: Alle Dokumente mit FolderId = 5 werden zurückgegeben

Testfall 2: DeleteDocument(id)

Testdaten: DocumentId = 10

Erwartung: Dokument wird aus DB gelöscht

SearchService

Testfall 1: SearchDocuments("Rechnung")

Testdaten: PlainText enthält "Rechnung" in 3 Dokumenten

Erwartung: 3 Dokumente werden zurückgegeben

Testfall 2: SearchDocuments("xyz123")

Testdaten: Kein Dokument enthält "xyz123"

Erwartung: Leere Liste

ImportService

Testfall 1: ValidateFileType("test.pdf")

Testdaten: Pfad mit .pdf Extension

Erwartung: true

Testfall 2: ValidateFileType("test.txt")

Testdaten: Pfad mit .txt Extension

Erwartung: false

TextExtractor

Testfall 1: ExtractFromPdf(pdfBytes)

Testdaten: PDF mit Text "Hallo Welt"

Erwartung: String "Hallo Welt" wird zurückgegeben

Testfall 2: ExtractFromDocx(docxBytes)

Testdaten: DOCX mit Text "Test 123"

Erwartung: String "Test 123" wird zurückgegeben

FileWatcher

Testfall 1: Start() + neue Datei erstellen

Testdaten: Neue "test.pdf" in überwachtem Ordner erstellen

Erwartung: OnFileCreated wird aufgerufen, Dokument wird importiert

Testfall 2: Stop()

Testdaten: FileWatcher läuft, Stop() wird aufgerufen

Erwartung: FileSystemWatcher wird gestoppt, keine Events mehr

MainWindow

Testfall 1: LoadFolderTree()

Testdaten: 3 Root-Ordner in DB

Erwartung: TreeView zeigt 3 Root-Ordner

Testfall 2: OnSearchClick mit "test"

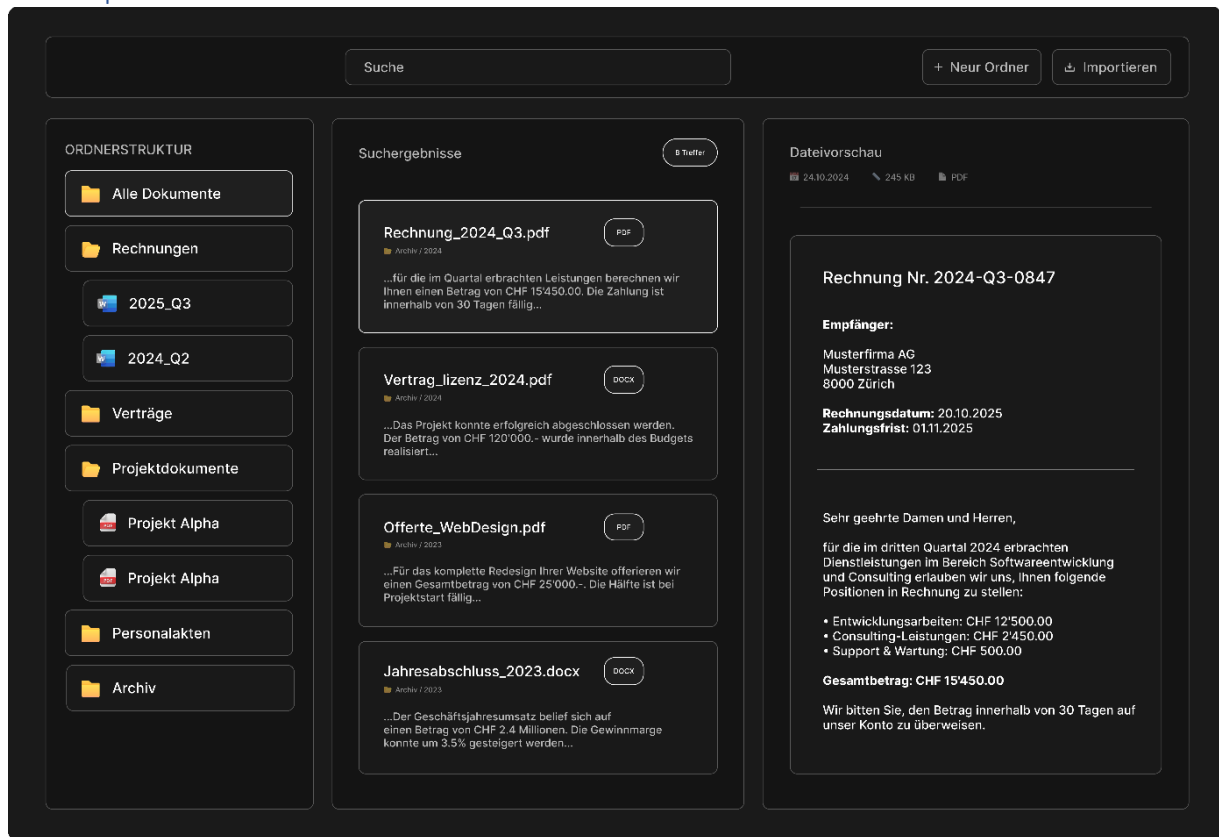
Testdaten: Suchbegriff "test"

Erwartung: SearchService wird aufgerufen, Ergebnisse werden in ListView angezeigt

Exp: Realistisches Mockup erstellen & Exp: Zustände der Masken geplant

Benutzeroberfläche – Mockup (D7: realistisches Mockup erstellen)

Mockup GUI



GUI-Zustände - Archivsoftware (D8: Zustände der Masken geplant)

Zustand 1: Start / Leer

Wann: Programm gestartet, keine Dokumente vorhanden

Bereich	Zustand	Element
Suchfeld	Aktiv	Leer, Placeholder "Suche"
Button "Neuer Ordner"	Aktiv	Klickbar
Button "Importieren"	Aktiv	Klickbar
Ordnerstruktur (Links)	Leer	"Alle Dokumente" als Root-Ordner
Suchergebnisse (Mitte)	Leer	Keine Treffer angezeigt
Dateivorschau (Rechts)	Leer	Keine Datei ausgewählt

Meldung: "Keine Dokumente vorhanden" (in der Mitte)

Zustand 2: Normal / Dokumente vorhanden

Wann: Dokumente sind importiert, kein Filter aktiv

Bereich	Zustand	Element
Suchfeld	Aktiv	Leer, bereit für Eingabe
Button "Neuer Ordner"	Aktiv	Klickbar
Button "Importieren"	Aktiv	Klickbar
Ordnerstruktur (Links)	Gefüllt	Ordner angezeigt, auf-/zuklappbar
Suchergebnisse (Mitte)	Leer	Keine aktive Suche
Dateivorschau (Rechts)	Leer/Gefüllt	Abhängig von Auswahl

Hinweis: User kann Ordner oder Dokument auswählen

Zustand 3: Suche aktiv / Treffer vorhanden

Wann: Suchbegriff eingegeben, Treffer gefunden

Bereich	Zustand	Element
Suchfeld	Aktiv	Suchbegriff sichtbar (z.B. "Rechnung")
Button "Neuer Ordner"	Aktiv	Klickbar
Button "Importieren"	Aktiv	Klickbar
Ordnerstruktur (Links)	Aktiv	Normal angezeigt
Suchergebnisse (Mitte)	Gefüllt	Liste mit Treffern + Badge "8 Treffer"
Dateivorschau (Rechts)	Leer/Gefüllt	Abhängig ob Treffer angeklickt

Treffer-Anzeige:

- Dateiname + Icon (PDF/DOCX)
- Pfad (Archiv / 2024)
- Snippet mit Suchbegriff hervorgehoben
- Badge "8 Treffer" oben rechts

Zustand 4: Suche aktiv / Keine Treffer

Wann: Suchbegriff eingegeben, keine Treffer

Bereich	Zustand	Element
Suchfeld	Aktiv	Suchbegriff sichtbar (z.B. "xyz123")
Button "Neuer Ordner"	Aktiv	Klickbar
Button "Importieren"	Aktiv	Klickbar
Ordnerstruktur (Links)	Aktiv	Normal angezeigt
Suchergebnisse (Mitte)	Leer	Badge "0 Treffer"
Dateivorschau (Rechts)	Leer	Keine Auswahl

Meldung: "Keine Treffer gefunden" (Mitte)

Zustand 5: Dokument ausgewählt

Wann: User klickt auf Dokument in Liste oder Baum

Bereich	Zustand	Element
Suchfeld	Aktiv	Suchbegriff oder leer
Button "Neuer Ordner"	Aktiv	Klickbar
Button "Importieren"	Aktiv	Klickbar
Ordnerstruktur (Links)	Aktiv	Ausgewähltes Dokument highlighted
Suchergebnisse (Mitte)	Aktiv	Ausgewählter Eintrag highlighted
Dateivorschau (Rechts)	Gefüllt	Dokumentinhalt angezeigt

Dateivorschau zeigt:

- Dokumenttitel (z.B. "Rechnung Nr. 2024-Q3-0847")
- Metadaten (Datum: 24.10.2024, Grösse: 245 KB, Typ: PDF)
- Volltext-Inhalt des Dokuments
- Bei aktiver Suche: Suchbegriff gelb markiert

Zustand 6: Import läuft

Wann: Datei wird gerade importiert (manuell oder Watcher)

Bereich	Zustand	Element
Suchfeld	Deaktiviert	Grau ausgegraut
Button "Neuer Ordner"	Deaktiviert	Grau ausgegraut
Button "Importieren"	Deaktiviert	Grau ausgegraut
Ordnerstruktur (Links)	Deaktiviert	Keine Interaktion möglich
Suchergebnisse (Mitte)	-	-
Dateivorschau (Rechts)	-	-

Zustand 7: Fehler / Validierung

Wann: User-Eingabe ungültig oder Fehler aufgetreten

Fall A: Leere Suche

Element	Zustand
Suchfeld	Rot umrandet
Fehlermeldung	"Bitte Suchbegriff eingeben"

Fall B: Ordner-Name existiert bereits

Element	Zustand
Dialog	Modal-Dialog geöffnet
Fehlermeldung	"Ordner 'Rechnungen' existiert bereits"
Button	"OK" zum Schliessen

Fall C: Falscher Dateityp

Element	Zustand
Dialog	Modal-Dialog geöffnet
Fehlermeldung	"Nur PDF/DOCX erlaubt"
Button	"OK" zum Schliessen

Fall D: Ordner löschen mit Dokumenten

Element	Zustand
Dialog	Warnungs-Dialog geöffnet
Warnung	"Ordner enthält Dokumente. Wirklich löschen?"
Buttons	"Ja" / "Abbrechen"

Zustand 8: Ordner aufklappen/zuklappen

Wann: User klickt auf Pfeil bei Ordner

Bereich	Zustand	Element
Ordnerstruktur (Links)	Animiert	Ordner klappt auf/zu
Pfeil-Icon	Gedreht	► (zugeklappt) / ▼ (aufgeklappt)
Unterordner	Sichtbar/Versteckt	Ein-/Ausgeblendet

Alle anderen Bereiche: Unverändert

Zustand 9: Fenster vergrößert/verkleinert

Wann: User ändert Fenstergrösse

Bereich	Verhalten
Alle Bereiche	Responsive Layout
Ordnerstruktur	Breite bleibt fix (300px)
Suchergebnisse	Breite passt sich an
Dateivorschau	Breite passt sich an

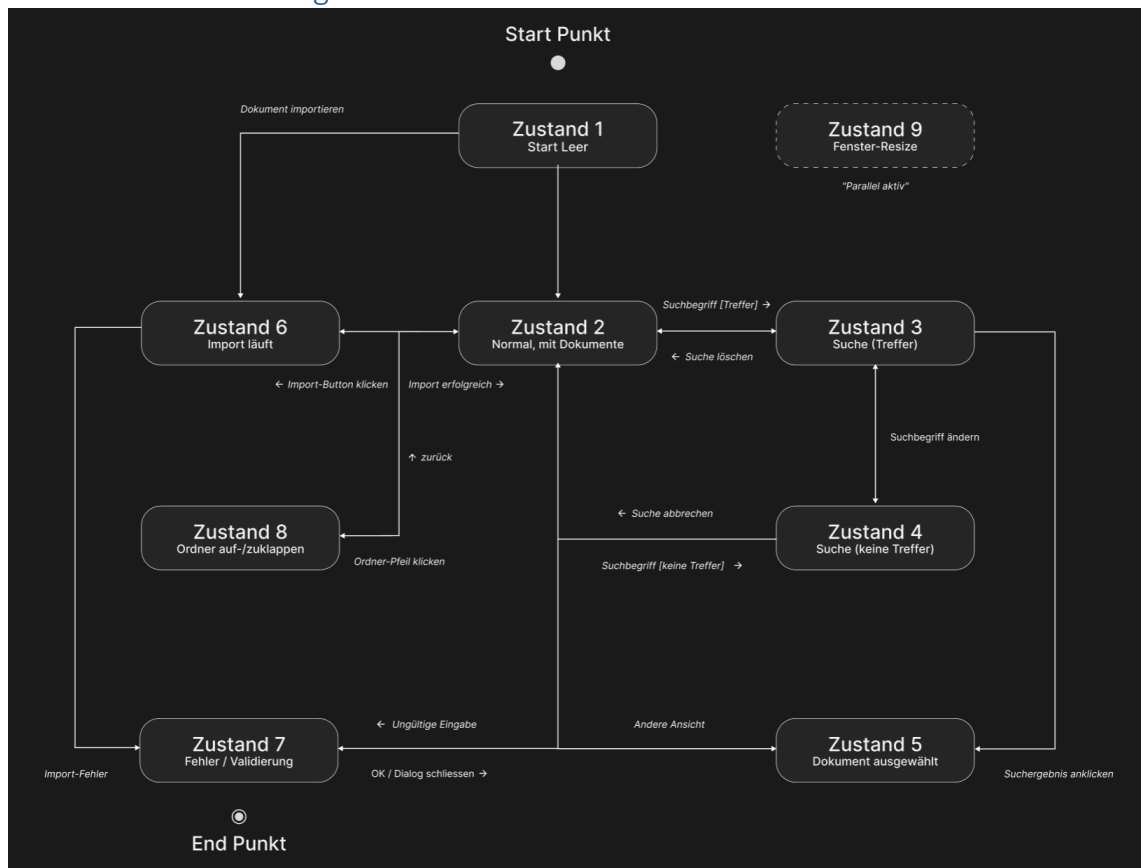
Minimum-Breite: 1024px

Zusammenfassung

Insgesamt 9 definierte Zustände:

1. Start / Leer
2. Normal / Dokumente vorhanden
3. Suche aktiv / Treffer
4. Suche aktiv / Keine Treffer
5. Dokument ausgewählt
6. Import läuft
7. Fehler / Validierung (4 Varianten)
8. Ordner auf-/zuklappen
9. Fenster-Resize

Zustände Grafisch dargestellt



Übergeordnete Themen

Einsatz von Komponenten von Drittherstellern wird dokumentiert (AB1)

In unserem Projekt setzen wir mehrere NuGet-Pakete ein, um die Kernfunktionalität zu realisieren.

Übersicht verwendeter Pakete

Paket	Version	Zweck im Projekt
DocumentFormat.OpenXml	3.3.0	Text aus DOCX-Dateien extrahieren
itext7	9.4.0	Text aus PDF-Dateien extrahieren
Microsoft.EntityFrameworkCore	8.0.0	ORM für Datenbankzugriff
Microsoft.EntityFrameworkCore.SqlServer	8.0.0	SQL Server Provider für EF Core
Microsoft.EntityFrameworkCore.Tools	8.0.0	Migrations und Scaffolding
Microsoft.Data.SqlClient	7.0.0-preview	SQL Server Verbindung

Verwendung der Komponenten

1. DocumentFormat.OpenXml

- **Hersteller:** Microsoft (Open Source, MIT License)
- **Verwendung:** Extraktion von Text aus Word-Dokumenten (.docx)
- **Implementierung:** DocxTextExtractor.cs im Business Layer
- **Warum:** Offizielles Microsoft-Paket, kostenlos, stabile API

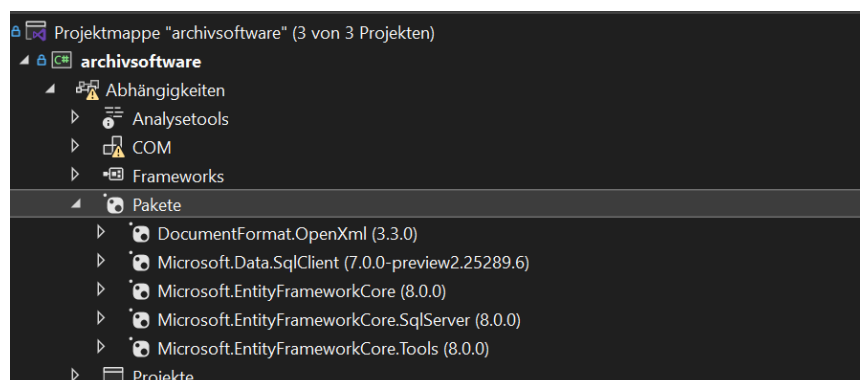
2. itext7

- **Hersteller:** iText Group NV (AGPL License)
- **Verwendung:** Extraktion von Text aus PDF-Dokumenten
- **Implementierung:** PdfTextExtractor.cs im Business Layer
- **Warum:** Moderne Version mit besserer Performance als alte iTextSharp (5.x)

3. Entity Framework Core (inkl. SqlServer & Tools)

- **Hersteller:** Microsoft (Open Source, MIT License)
- **Verwendung:**
 - ORM für typsicheren Datenbankzugriff
 - SQL Server LocalDB Verbindung
 - **Implementierung:** ArchiveContext.cs im DataAccess Layer
 - **Warum:** Projektanforderung, Standard-ORM für .NET, LINQ-Support

4.



Microsoft.Data.SqlClient

- **Hersteller:** Microsoft (MIT License)
- **Verwendung:** Low-Level SQL Server Verbindung (intern von EF Core genutzt)
- **Warum:** Von Entity Framework Core 8.0 empfohlen

Architektur-Zuordnung

Presentation Layer (WPF):

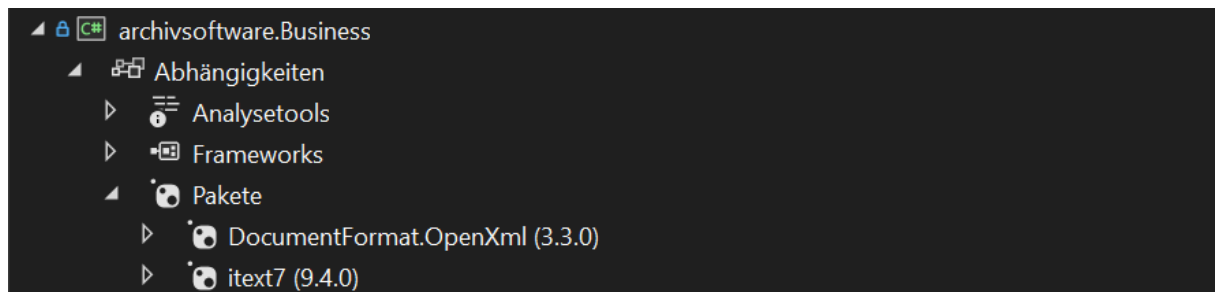
- Keine direkten Drittkomponenten

Business Layer:

- itext7 → PDF Text-Extraktion
- DocumentFormat.OpenXml → DOCX Text-Extraktion

Data Access Layer:

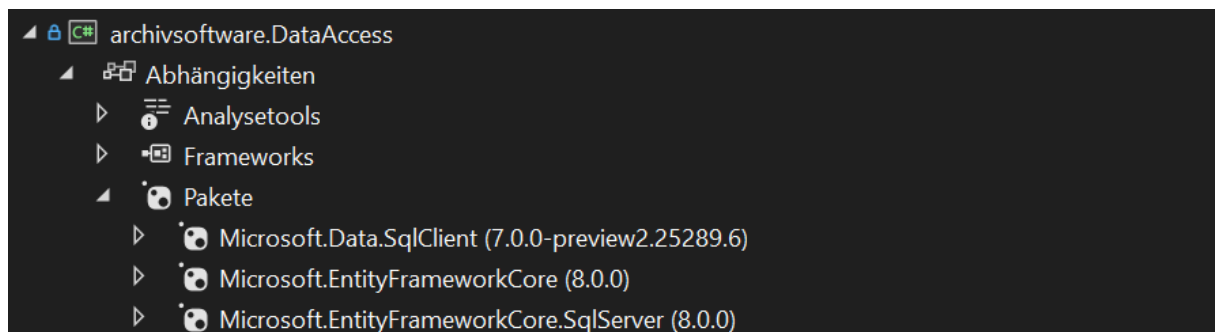
- Entity Framework Core → Datenbankzugriff
- SqlServer Provider → SQL Server Unterstützung
- SqlConnection → Verbindungsverwaltung



Installation

Alle Pakete wurden über den **NuGet Package Manager** in Visual Studio installiert:

- GUI-basiert: Rechtsklick auf Projekt → "NuGet-Pakete verwalten"
- Automatische Abhängigkeitsauflösung durch NuGet



Begründung der Auswahl

Ohne diese Libraries wäre das Projekt nicht realisierbar:

- Text-Extraktion aus PDF/DOCX manuell zu implementieren würde Wochen dauern
- Entity Framework Core war Projektanforderung (3-Schichten-Architektur mit EF Core)
- Alle gewählten Pakete sind Industrie-Standard und gut dokumentiert

Dateien lesen und schreiben (txt, csv, json) (AB3)

Testing mittels UnitTest (AB4)

Clean Code Regeln werden eingehalten und anhand eines Beispiels dokumentiert (AB4)

In unserem Projekt haben wir die wichtigsten Clean Code Prinzipien aus AB4 konsequent umgesetzt. Im Folgenden werden diese anhand konkreter Code-Beispiele aus *MainWindow.cs* demonstriert.

1. Naming Conventions (C# Coding Standards)

Wir halten uns an die Microsoft C# Naming Conventions:

```
// PascalCase für Klassen
public partial class MainWindow : Window

// _camelCase für private Felder
private readonly ArchiveContext _context;
private readonly FolderRepository _folderRepository;

// PascalCase für Methoden
private void LoadFolderTree()

// PascalCase für Properties
public ObservableCollection<FolderViewModel> FolderTree { get; set; }
```

Begründung: Einheitliche Namensgebung macht den Code für alle Entwickler sofort lesbar.

2. KISS-Prinzip (Keep It Simple, Stupid)

Wir verwenden **Early Return** um unnötige Verschachtelungen zu vermeiden:

```
private void BtnImport_Click(object sender, RoutedEventArgs e)
{
    // Früher Abbruch bei fehlendem Ordner
    if (_selectedFolder == null)
    {
        MessageBox.Show("Bitte wählen Sie zuerst einen Ziel-Ordner aus...");
        return;
    }

    // Hauptlogik erst wenn Validierung OK
    var openFileDialog = new Microsoft.Win32.OpenFileDialog
    {
        Filter = "Unterstützte Dokumente (*.pdf;*.docx)|*.pdf;*.docx",
        Multiselect = true
    };

    if (openFileDialog.ShowDialog() == true)
    {
        foreach (string filePath in openFileDialog.FileNames)
        {
            ImportDocument(filePath, _selectedFolder.Id);
        }
    }
}
```

Vorteil: Weniger Verschachtelung, bessere Lesbarkeit.

3. DRY-Prinzip (Don't Repeat Yourself)

Wiederverwendbare Methode für Dateigrößen-Formatierung:

```
private string FormatFileSize(long bytes)
{
    if (bytes < 1024)
        return $"{bytes} B";
    else if (bytes < 1024 * 1024)
        return $"{bytes / 1024} KB";
    else
        return $"{bytes / (1024 * 1024)} MB";
}
```

Verwendung an mehreren Stellen:

```
// In Dokumenten-Card
var metaText = new TextBlock {
    Text = $"{FormatFileSize(document.FileSize)} • {importDate}"
};

// In Metadaten-Anzeige
AddMetadataRow("Größe:", FormatFileSize(document.FileSize));
```

Vorteil: Bei Änderungen muss nur eine Stelle angepasst werden.

4. Single Responsibility Principle (SRP)

Jede Methode hat genau eine Aufgabe:

```
// LoadDocuments: Lädt Dokumente und aktualisiert UI
private void LoadDocuments(int folderId)
{
    var documents = _documentRepository.GetByFolderId(folderId);
    ResultsPanel.Children.Clear();
    ResultCountText.Text = $"{documents.Count} Dokument(e)";

    foreach (var document in documents.OrderByDescending(d => d.ImportedAt))
    {
        var card = CreateDocumentCard(document);
        ResultsPanel.Children.Add(card);
    }
}

// CreateDocumentCard: Erstellt nur die UI-Card
private Border CreateDocumentCard(Document document)
{
    var card = new Border
    {
        Background = new SolidColorBrush(Color.FromRgb(26, 26, 26)),
        BorderBrush = new SolidColorBrush(Color.FromRgb(42, 42, 42)),
        BorderThickness = new Thickness(1),
        CornerRadius = new CornerRadius(8)
    };
    // ... weitere UI-Elemente
    return card;
}
```

Begründung: Trennung von Daten-Logik und UI-Erstellung erleichtert Testing und Wartung.

5. Fehlerbehandlung mit Try-Catch

Alle kritischen Operationen sind mit Fehlerbehandlung abgesichert:

```
private void ImportDocument(string filePath, int folderId)
{
    try
    {
        byte[] fileData = File.ReadAllBytes(filePath);
        string fileExtension = Path.GetExtension(filePath).ToLower();

        // Text extrahieren je nach Dateityp
        string plainText = string.Empty;
        if (fileExtension == ".pdf")
        {
            var pdfExtractor = new PdfTextExtractor();
            plainText = pdfExtractor.ExtractText(fileData);
        }
        else if (fileExtension == ".docx")
        {
            var docxExtractor = new DocxTextExtractor();
            plainText = docxExtractor.ExtractText(fileData);
        }

        // Dokument in DB speichern
        var document = new Document
        {
            FileName = Path.GetFileName(filePath),
            FileType = fileExtension,
            FileData = fileData,
            PlainText = plainText,
            FolderId = folderId
        };

        _documentRepository.Add(document);
        _documentRepository.SaveChanges();
    }
    catch (FileNotFoundException ex)
    {
        MessageBox.Show($"Datei nicht gefunden: {ex.Message}");
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Fehler beim Import: {ex.Message}");
    }
}
```

Wichtig: Spezifische Exceptions (FileNotFoundException) vor generischem Exception abfangen.

6. XML-Dokumentations-Kommentare

Wichtige Methoden sind mit XML-Kommentaren dokumentiert:

```
/// <summary>
/// Lädt die Ordnerstruktur aus der Datenbank (hierarchisch für TreeView)
/// </summary>
private void LoadFolderTree()
{
    try
    {
        var rootFolders = _folderRepository.GetRootFolders();
        _folderTree = new ObservableCollection<FolderViewModel>();

        foreach (var folder in rootFolders)
        {
            var folderVM = new FolderViewModel(folder);
            LoadSubFolders(folderVM);
            _folderTree.Add(folderVM);
        }

        FolderTreeView.ItemsSource = _folderTree;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Fehler beim Laden: {ex.Message}");
    }
}

/// <summary>
/// Rekursiv Unterordner laden
/// </summary>
private void LoadSubFolders(FolderViewModel parentVM)
{
    var subFolders = _folderRepository.GetSubFolders(parentVM.Id);
    foreach (var subFolder in subFolders)
    {
        var subFolderVM = new FolderViewModel(subFolder);
        LoadSubFolders(subFolderVM);
        parentVM.AddChild(subFolderVM);
    }
}
```

Zusammenfassung

Prinzip	Umsetzung im Projekt
Naming Conventions	PascalCase für Klassen/Methoden, _camelCase für Felder
KISS	Early Return Pattern, klare Strukturen
DRY	FormatFileSize() wiederverwendet
SRP	Klare Trennung: Laden, Erstellen, Anzeigen
Try-Catch	Alle Datei-/DB-Operationen abgesichert
XML-Kommentare	Wichtige Methoden dokumentiert

Fehlerhandling (try catch) sinnvoll eingesetzt

API Dokumentation (DoxFX, Sandcastle, DoxyGen)